

F.Y. M.Sc. Computer Science | Semester II

MOBILE APP DEVELOPMENT TECHNOLOGIES

CS-552 MJ

Exam Coverage: 2023 Pattern | Max Marks: 70 | Time: 3 Hours

Prepared by: Aditya Bapusaheb Chaudhari, Full Stack Engineer
Contact Details

LinkedIn: <https://www.linkedin.com/in/aditya-chaudhari0301/>

GitHub: <https://github.com/chaudhari-aditya03>

Email: aditya.chaudhari.dev@gmail.com

Document Overview & Utility

This Study Guide consists of 10 Structured Units, covering core topics: Android Architecture, Components (Activity, Service), UI/Layouts, Intents, Multithreading, Data Storage, Location Services, and Hybrid Apps (PhoneGap/Cordova).

Benefits & Exam Readiness:

High-Impact Focus: Frequently tested and high-priority concepts.

- **Clarity and Structure:** All technical syntax, examples, and model answers are presented in a clean, simplified, and numbered format, ideal for last-minute revision.
 - **Ready for Distribution:** This guide provides a complete, professionally formatted resource designed to ensure every student achieves maximum exam preparedness.
 - **Stay Connected:** Students are encouraged to connect with the author, Aditya Chaudhari, for updates and networking via My LinkedIn Profile.
-

UNIT 1: Introduction to Mobile Applications & Android

1.1 Types of Mobile Applications

Mobile applications are classified into three main types based on how they are built and delivered:

Type	Description	Examples	Pros / Cons
Native Apps	Built specifically for one platform using platform-specific language	Instagram (iOS), WhatsApp	Fast & full access to device features / Expensive to build for multiple platforms
Web Apps	Run in mobile browser, built with HTML/CSS/JS	Gmail Web, Twitter Web	Write once run anywhere / Limited device feature access, needs internet
Hybrid Apps	Wrapped web app inside a native container (e.g., PhoneGap)	Apache Cordova apps	Cross-platform, cheaper / Slower than native, not full native feel

1.2 Mobile Operating Systems

Key Mobile Operating Systems and their characteristics:

- Android (Google) - Open-source, Linux-based, most widely used globally, supports Java/Kotlin development
- iOS (Apple) - Closed ecosystem, UNIX-based, highly secure, uses Swift/Objective-C, runs only on Apple devices
- Windows Phone (Microsoft) - Discontinued, was based on Windows NT kernel
- BlackBerry OS - Focus on security and enterprise, largely discontinued
- Symbian (Nokia) - Pioneer mobile OS, now discontinued
- HarmonyOS (Huawei) - Newer OS developed as an alternative to Android

Note: Android dominates with ~72% global market share. Exam often asks to list and differentiate mobile OS.

1.3 Factors in Developing Mobile Applications

Key factors a developer must consider:

1. Screen Size & Resolution - Different devices have different screen sizes; use responsive layouts
2. Performance - Mobile devices have limited CPU, RAM; optimize code and resources
3. Battery Life - Minimize background processing and location updates to save battery
4. Network Connectivity - Handle poor/no network conditions gracefully; support offline mode
5. Security - Encrypt sensitive data; use secure APIs; implement proper permissions
6. User Interface (UI) - Design for touch input; keep UI simple and intuitive
7. Platform Compatibility - Support multiple Android versions; test on different devices
8. Storage Limitations - Manage internal/external storage wisely; compress assets
9. Monetization Strategy - Decide between paid, freemium, ads-based models
10. Testing & Deployment - Test on emulators and real devices; follow Play Store guidelines

1.4 Applications of Android

Application Area	Description	Examples
1. Communication	Voice calls, messaging, video conferencing	WhatsApp, Skype, Google Meet, Zoom
2. Social Media	Sharing posts, photos, videos, networking	Facebook, Instagram, Twitter, LinkedIn
3. E-Commerce	Online shopping and payments	Amazon, Flipkart, Google Pay, PhonePe
4. Navigation & Maps	GPS navigation, real-time traffic, location services	Google Maps, Waze, Ola, Uber
5. Healthcare	Patient monitoring, telemedicine, health tracking	mHealth apps, Heart rate monitor, Fitbit
6. Education	e-Learning, video lectures, online exams	YouTube, Coursera, BYJU's, Duolingo
7. Entertainment	Streaming music, video, gaming	Netflix, Spotify, PUBG, Candy Crush
8. Banking & Finance	Mobile banking, stock trading, insurance	SBI YONO, Zerodha, PolicyBazaar
9. Smart Home / IoT	Control home devices via smartphone	Google Home, SmartThings, Alexa app
10. Productivity	Office tools, cloud sync, task management	Gmail, Drive, MS Office, Notion

Note: Android's open-source nature and massive developer ecosystem make it suitable for virtually every domain of mobile computing.

UNIT 2: Android Architecture & Core Concepts

2.1 Android Architecture (Layered Architecture)

Android architecture consists of 5 layers from bottom to top:

Layer 1: Linux Kernel (Bottom)

- The foundation of Android OS
- Manages hardware drivers: Camera, Display, Bluetooth, WiFi, Audio
- Handles core OS functions: Memory management, Process management, Security, Power management
- Android uses a modified version of the Linux kernel

Layer 2: Hardware Abstraction Layer (HAL)

- Bridge between hardware drivers (Linux Kernel) and the Android framework
- Provides standard interfaces so higher layers do not depend on hardware specifics

Layer 3: Android Runtime (ART) & Native Libraries

- ART (Android Runtime) - Executes Android apps; replaced Dalvik from Android 5.0+
- Dalvik Virtual Machine - Older runtime, used register-based bytecode (.dex files)
- Native Libraries (C/C++) - OpenGL, SQLite, WebKit, SSL, libc, Media Framework

Layer 4: Application Framework (API Layer)

- Activity Manager - Manages app lifecycle and back stack
- Content Providers - Share data between apps
- Resource Manager - Manages non-code resources (layouts, strings, drawables)
- Notification Manager - Manages system notifications
- Location Manager - Provides location and GPS services
- Telephony Manager - Access phone call functions
- View System - Build UI components

Layer 5: Applications (Top Layer)

- Pre-installed system apps: Phone, Contacts, Browser, Camera, Settings
- Third-party apps installed from Google Play Store

Note: Exam tip: Draw a layered diagram and label all 5 layers. This is asked in Q3/Q4 frequently.

2.2 Dalvik Virtual Machine (DVM)

Dalvik Virtual Machine is a register-based virtual machine designed specifically for Android:

- Developed by Dan Bornstein at Google
- Executes .dex (Dalvik Executable) files, which are compiled from Java .class files
- Optimized for low memory and slow CPUs found in mobile devices
- Each app runs in its own process with its own Dalvik VM instance for security/isolation
- Replaced by ART (Android Runtime) from Android 5.0 (Lollipop) onwards
- ART uses AOT (Ahead-Of-Time) compilation, while Dalvik used JIT (Just-In-Time)

2.3 Android Manifest File

The AndroidManifest.xml is a critical configuration file present at the root of every Android project. It provides essential information about the app to the Android system.

Key Contents of Manifest File:

- Package name - Unique identifier for the app (e.g., com.example.myapp)
- App Components - Declares all Activities, Services, Broadcast Receivers, Content Providers
- Permissions - Declares required permissions (e.g., INTERNET, CAMERA, READ_CONTACTS)
- Min & Target SDK Version - Specifies Android version compatibility
- App Icon & Label - References to app name and icon resources
- Intent Filters - Specifies what intents each component can handle

```
<!-- Sample AndroidManifest.xml structure -->
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">

    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>

    <application
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name">

        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>

        <service android:name=".MyService"/>
        <receiver android:name=".MyReceiver"/>
    </application>
</manifest>
```

2.4 Android Virtual Device (AVD)

An Android Virtual Device (AVD) is an emulator configuration that lets you run an Android environment on your computer:

- Created using AVD Manager in Android Studio
 - Simulates a real Android device with configurable hardware profiles
 - Can emulate different screen sizes, Android versions, RAM, storage
 - Useful for testing apps without a physical device
 - Slower than real devices; use physical device for performance testing
 - AVD runs in QEMU-based emulator provided with Android SDK
-

UNIT 3: Android Application Components

3.1 Components of an Android Application

Android applications are built from four fundamental components:

1. Activity

- Represents a single screen with a user interface
- Each activity is an independent screen; apps can have multiple activities
- Entry point for user interaction
- Example: LoginActivity, MainActivity, ProfileActivity

2. Service

- Runs in the background without a UI
- Types: Started Service, Bound Service, Foreground Service
- Used for: Playing music, downloading files, syncing data
- Example: MusicService, DownloadService

3. Broadcast Receiver

- Listens for system-wide broadcast announcements
- Examples of broadcasts: BOOT_COMPLETED, BATTERY_LOW, SMS_RECEIVED, CONNECTIVITY_CHANGED
- Registered in Manifest (static) or at runtime (dynamic)
- Does not have a UI but can display notifications or start activities

4. Content Provider

- Manages shared data between applications
- Provides a standard interface for data access using URIs
- Examples: Contacts app, Media Store, Calendar
- Built on top of SQLite database usually
- Uses CRUD operations: query(), insert(), update(), delete()

3.2 Activity Lifecycle

Every Activity goes through a well-defined lifecycle managed by the Android system:

Callback Method	When Called	Key Action
onCreate()	Activity is first created	Initialize UI, set content view, bind data
onStart()	Activity becomes visible to user	Start animations, register receivers
onResume()	Activity starts interacting with user (foreground)	Resume paused tasks, acquire camera/mic
onPause()	Another activity comes to foreground	Pause animations, save data, release resources
onStop()	Activity is no longer visible	Release heavy resources, save app state
onRestart()	Activity returning from stopped state	Refresh UI data if needed
onDestroy()	Activity is being destroyed	Clean up all resources, unregister receivers

Note: Lifecycle order: `onCreate` -> `onStart` -> `onResume` -> `[Running]` -> `onPause` -> `onStop` -> `onDestroy`

3.3 Fragment Lifecycle

A Fragment is a reusable portion of UI that can be embedded in an Activity. Fragments have their own lifecycle:

- `onAttach()` - Fragment attaches to Activity
- `onCreate()` - Fragment is created
- `onCreateView()` - Fragment's UI is created (inflate layout here)
- `onActivityCreated()` - Activity's `onCreate()` is complete
- `onStart()` - Fragment becomes visible
- `onResume()` - Fragment is interactive
- `onPause()` - Fragment is no longer interactive
- `onStop()` - Fragment is no longer visible
- `onDestroyView()` - Fragment's view is being destroyed
- `onDestroy()` - Fragment is destroyed
- `onDetach()` - Fragment detaches from Activity

Note: Fragments allow flexible, multi-pane UI on tablets. Fragment manager handles fragment transactions.

3.4 Android Services and Lifecycle

A Service is an app component that runs in the background to perform long-running operations.

Types of Services

- **Started Service:** Runs indefinitely until `stopSelf()` or `stopService()` is called.
- **Bound Service:** Runs as long as a component is bound to it. Allows two-way communication.
- **Foreground Service:** Runs with a persistent notification shown to the user.
- **Intent Service:** Handles asynchronous requests in a worker thread; auto-stops when done. (Deprecated in API 30).

C.1 Started Service Lifecycle (`startService()`)

When a component calls `startService()`, the service runs independently through the following steps:

1. `onCreate()`: Called when the service is first created. Use this for one-time setup.
2. `onStartCommand(Intent, flags, startId)`: Called every time `startService()` is called. This is where the actual work is done.
3. **Service Running:** The service continues to run until it stops itself (`stopSelf()`) or another component stops it (`stopService()`).
4. `onDestroy()`: Called when the service is stopped and destroyed. Use this to clean up resources.

Note: A started service continues running even if the component that started it is destroyed (e.g., app goes to background).

C.2 Bound Service Lifecycle (`bindService()`)

When a component binds to a service to interact with it:

1. `onCreate()`: Called when service is first created
2. `onBind(Intent)`: Called when a component binds. Returns an `IBinder` interface for communication.
3. **Service Bound:** Service stays alive as long as at least one component is bound

4. `onUnbind(Intent)`: Called when all components have unbound
5. `onDestroy()`: Called when service is destroyed

C.3 Return Values of `onStartCommand()`

Return Value	Meaning
START_STICKY	If killed, system restarts the service with null intent. Good for media playback.
START_NOT_STICKY	If killed, system does NOT restart. Good for tasks that should complete or be retriggered.
START_REDELIVER_INTENT	If killed, system restarts with the last intent. Good for download tasks.

3.5 Broadcast Receiver

Broadcast Receivers respond to system-wide broadcast announcements or custom broadcasts from apps:

```
// Registering in Manifest (Static Registration):
<receiver android:name=".MyReceiver">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED"/>
    </intent-filter>
</receiver>

// Dynamic Registration in Java:
IntentFilter filter = new IntentFilter(Intent.ACTION_BATTERY_LOW);
registerReceiver(myReceiver, filter);

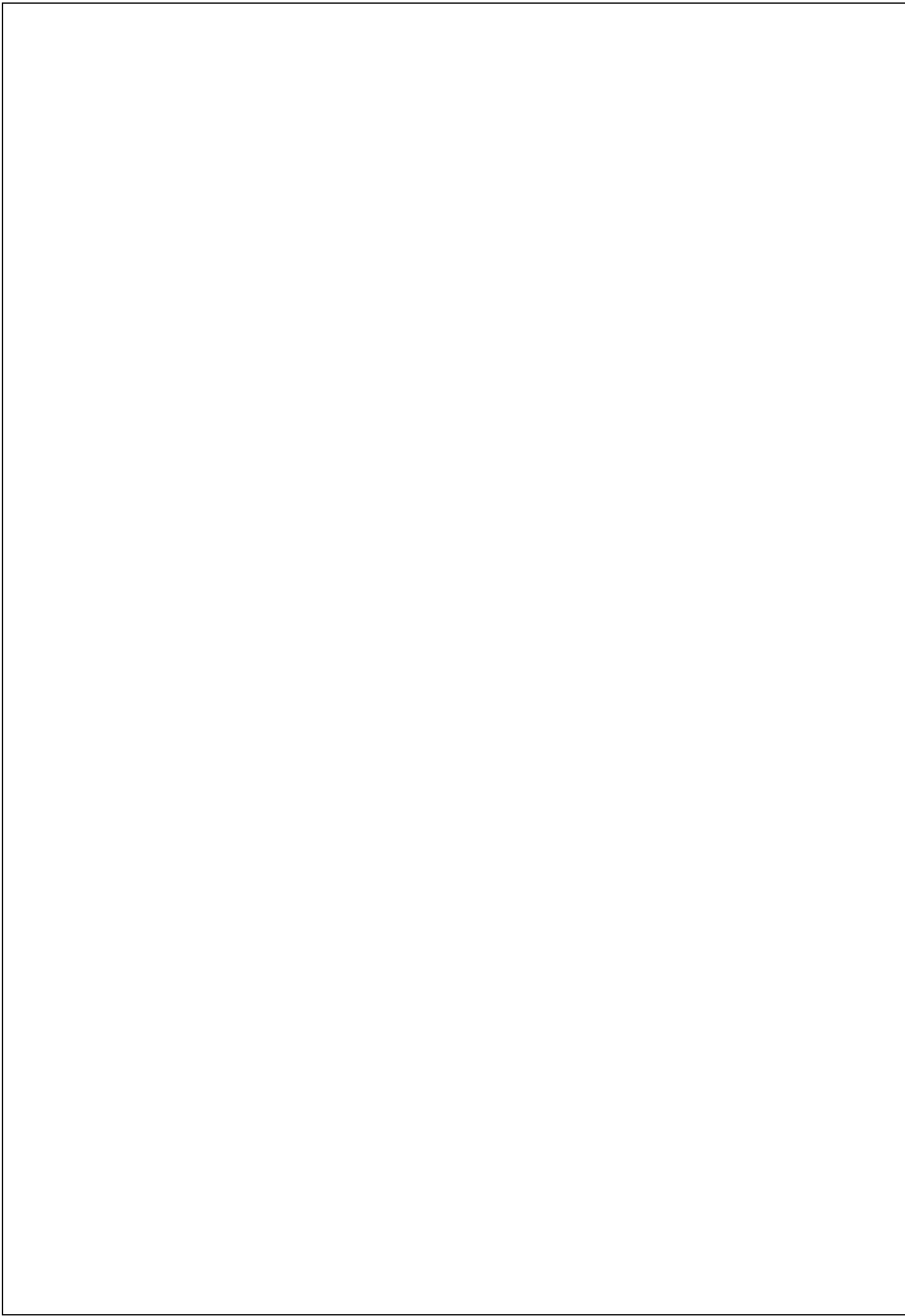
// Receiver Class:
public class MyReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "Broadcast Received!", Toast.LENGTH_SHORT).show();
    }
}
```

3.6 Content Providers

Content Providers manage access to structured data and provide a standard interface for sharing data across apps:

- Data is accessed via a URI (Uniform Resource Identifier)
- URI format: `content://authority/path/id` e.g., `content://contacts/people/1`
- Key methods: `query()`, `insert()`, `update()`, `delete()`, `getType()`
- Accessed via `ContentResolver` from another app

```
// Querying Contacts ContentProvider:
ContentResolver cr = getContentResolver();
Cursor cursor = cr.query(
    ContactsContract.Contacts.CONTENT_URI,
    null, null, null, null);
while(cursor.moveToNext()) {
    String name = cursor.getString(
        cursor.getColumnIndex(ContactsContract.Contacts.DISPLAY_NAME));
    Log.d("Contact", name);
}
```



UNIT 4: Android UI Components & Layouts

4.1 Android UI Controls

Android provides rich built-in UI controls (Views) for building user interfaces:

UI Control	Description	Common Attributes
TextView	Displays non-editable text	text, textSize, textColor
EditText	Input field for user text entry	hint, inputType, maxLength
Button	Clickable button to trigger actions	text, onClick
ImageView	Displays images	src, scaleType
CheckBox	Toggle on/off selection	checked, text
RadioButton	Single selection from a group	checked, text (use RadioGroup)
Spinner	Drop-down list for selection	entries, prompt
ListView	Scrollable list of items	adapter, divider
GridView	2D scrollable grid of items	numColumns, columnWidth
ProgressBar	Shows task progress	style, progress, max
SeekBar	Slider for selecting a value	max, progress
Switch	Toggle switch UI element	checked, text
RatingBar	Star-based rating widget	numStars, rating
WebView	Displays web pages inside app	loadUrl()

4.2 View and ViewGroup

1. View: The Basic UI Building Block

A View is the basic building block of the User Interface (UI) in Android. It represents a rectangular area on the screen responsible for:

- Drawing itself on the screen (rendering)
- Handling user interaction events (click, touch, key press)
- Every UI element visible on screen is a View or a subclass of View.

Definition: In Android, the `View` class is the superclass of all UI components. It is defined in the `android.view.View` package.

View Subclasses (UI Widgets):

View Subclass	What it displays
TextView	Displays text on screen
EditText	Editable text input field (subclass of TextView)
Button	Clickable button (subclass of TextView)
ImageView	Displays images and drawables
CheckBox	Two-state toggle (checked/unchecked)

View Subclass	What it displays
RadioButton	Single-choice within a RadioGroup
ProgressBar	Shows progress of an operation
SeekBar	Draggable slider control

Key Properties of View (XML Attributes):

- `android:id`: Unique identifier to find the view in code using `findViewById()`.
- `android:layout_width`: Width, typically `match_parent`, `wrap_content`, or fixed dp.
- `android:layout_height`: Height, typically `match_parent`, `wrap_content`, or fixed dp.
- `android:visibility`: `VISIBLE`, `INVISIBLE` (takes space), `GONE` (no space).
- `android:padding`: Internal spacing between view border and content.
- `android:margin`: External spacing between this view and other views.

Key Methods of View (Java/Kotlin):

1. `view.setVisibility(View.GONE);` // Hide the view
2. `view.setEnabled(false);` // Disable interaction
3. `view.setBackgroundColor(Color.RED);` // Change background color
4. `view.setOnClickListener(v -> { ... });` // Handle click events
5. `view.invalidate();` // Force redraw of the view

2. ViewGroup: The Container

A special View that can contain other Views (children) — also called a Layout. It defines how child views are arranged on the screen.

- Types of ViewGroups (Layouts): `LinearLayout`, `RelativeLayout`, `ConstraintLayout`, `FrameLayout`, `GridLayout`, `TableLayout`, `ScrollView`.

Note: A View is a single UI element. A ViewGroup is a container that holds multiple Views and defines their arrangement.

4.3 Types of Layouts

1. LinearLayout

Arranges child views in a single row (horizontal) or column (vertical):

- `android:orientation` - "horizontal" or "vertical"
- `android:layout_weight` - distributes remaining space among children

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">
    <TextView android:text="Name:" .../>
    <EditText android:hint="Enter name" .../>
    <Button android:text="Submit" .../>
</LinearLayout>
```

2. RelativeLayout

Positions children relative to each other or to the parent container:

- `android:layout_below`, `android:layout_toRightOf`, `android:layout_alignParentTop` etc.
- More flexible than `LinearLayout` but deprecated in favor of `ConstraintLayout`

3. ConstraintLayout

- Modern, flat layout that allows complex UIs without nesting
- Constraints connect views to each other or to the parent using guidelines
- Most performant layout for complex UIs

4. FrameLayout

- Stacks children on top of each other (last child drawn on top)
- Used for fragments, image overlays

5. GridLayout

- Arranges children in a rectangular grid
- Specify rowCount and columnCount

6. TableLayout

- Arranges views in rows and columns like an HTML table
- Use TableRow to define each row

4.4 Toast

A Toast is a simple feedback message that appears briefly on screen and then disappears automatically:

```
// Basic Toast:
Toast.makeText(this, "Hello World!", Toast.LENGTH_SHORT).show();

// Toast with Long duration:
Toast.makeText(this, "File Saved!", Toast.LENGTH_LONG).show();

// Custom positioned Toast:
Toast toast = Toast.makeText(this, "Custom Toast", Toast.LENGTH_SHORT);
toast.setGravity(Gravity.TOP|Gravity.CENTER_HORIZONTAL, 0, 0);
toast.show();
```

- LENGTH_SHORT - Shows for about 2 seconds
- LENGTH_LONG - Shows for about 3.5 seconds
- Toast does NOT pause app execution (non-blocking)

4.5 Spinner (Drop-down List)

Spinner provides a drop-down list for selecting one item from a set:

```
<!-- activity_main.xml -->
<Spinner
    android:id="@+id/spinner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>

// MainActivity.java
Spinner spinner = findViewById(R.id.spinner);
String[] subjects = {"Android", "Java", "Python", "Database", "Networks"};
ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
    android.R.layout.simple_spinner_item, subjects);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
spinner.setAdapter(adapter);

spinner.setOnItemClickListener(new AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int position, long
id) {
        String selected = parent.getItemAtPosition(position).toString();
        Toast.makeText(MainActivity.this, "Selected: " + selected,
Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onNothingSelected(AdapterView<?> parent) {}
});
```

4.6 Adapters

Adapters act as a bridge between a data source and a UI component like ListView, GridView, or Spinner:

- ArrayAdapter - Works with arrays or List of objects
- BaseAdapter - Most flexible; extend this for custom adapters
- CursorAdapter - Works with database Cursors
- SimpleAdapter - Maps data from a List of Maps to views
- RecyclerView.Adapter - Modern, efficient replacement for ListView adapter

4.7 Menus in Android

Android supports three types of menus:

1. Options Menu

- Appears in the app bar (action bar) when user taps the overflow icon
- Created by overriding onCreateOptionsMenu() and handling clicks in onOptionsItemSelected()

2. Context Menu

- Appears when user long-presses on a view
- Registered with registerForContextMenu(view); defined in onCreateContextMenu()

3. Popup Menu

- Appears anchored below or above a specific view on user action
 - Created with new PopupMenu(context, anchorView)
-

UNIT 5: Intents, Event Handling & Notifications

5.1 Intents in Android

An Intent is a messaging object used to request an action from another component:

Type	Description	Example
Explicit Intent	Specifies exact component (Activity/Service) to launch by class name	Start a specific Activity: <code>new Intent(this, LoginActivity.class)</code>
Implicit Intent	Declares general action; system finds suitable component to handle it	Open URL in browser, share text, make phone call, open camera

```
// Explicit Intent - Start another Activity:
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
intent.putExtra("key", "value"); // pass data
startActivity(intent);
```

```
// Implicit Intent - Open a URL:
Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("https://www.google.com"));
startActivity(intent);
```

```
// Implicit Intent - Make a call:
Intent intent = new Intent(Intent.ACTION_CALL);
intent.setData(Uri.parse("tel:9876543210"));
startActivity(intent);
```

5.2 Event Handling in Android

Event handling lets apps respond to user interactions (click, touch, long press, etc.):

Methods of Event Handling:

- Using Anonymous Inner Class - Define listener directly while setting it
- Implementing Listener Interface - Activity implements the listener interface
- Using XML onClick attribute - Specify method name in XML; method in Activity

```
// Method 1: Anonymous Inner Class
Button btn = findViewById(R.id.button);
btn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Toast.makeText(MainActivity.this, "Clicked!", Toast.LENGTH_SHORT).show();
    }
});
```

```
// Method 2: Lambda (Java 8+)
btn.setOnClickListener(v -> Toast.makeText(this, "Clicked!",
    Toast.LENGTH_SHORT).show());
```

```
// Common Event Listeners:
// View.OnClickListener - single tap
// View.OnLongClickListener - long press
// View.OnTouchListener - raw touch events
// TextWatcher - text changes in EditText
// AdapterView.OnItemClickListener - item selection in Spinner/ListView
```

5.3 Touch Events & Gesture Methods

Android provides gesture detection for complex touch interactions:

- `onDown()` - Called when user first touches the screen
- `onSingleTapUp()` - Single tap detected
- `onDoubleTap()` - Double tap detected
- `onLongPress()` - Finger held down for a long duration
- `onFling()` - Quick swipe movement
- `onScroll()` - Scrolling motion while finger is down
- `onShowPress()` - User has pressed, before any movement

```
GestureDetector gestureDetector = new GestureDetector(this,
    new GestureDetector.SimpleOnGestureListener() {
        @Override
        public boolean onFling(MotionEvent e1, MotionEvent e2,
                                float velocityX, float velocityY) {
            Toast.makeText(MainActivity.this, "Fling!", Toast.LENGTH_SHORT).show();
            return true;
        }
    });
```

5.4 Android Notifications

Notifications alert users to events outside the app. They appear in the status bar and notification drawer:

```
// Step 1: Create Notification Channel (required for API 26+)
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
    NotificationChannel channel = new NotificationChannel(
        "channel_id", "My Channel", NotificationManager.IMPORTANCE_DEFAULT);
    NotificationManager manager = getSystemService(NotificationManager.class);
    manager.createNotificationChannel(channel);
}

// Step 2: Build the Notification
NotificationCompat.Builder builder = new NotificationCompat.Builder(this,
    "channel_id")
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("My Notification")
    .setContentText("This is the notification body")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT);

// Step 3: Show the Notification
NotificationManagerCompat notificationManager =
    NotificationManagerCompat.from(this);
notificationManager.notify(1, builder.build());
```

5.5 Alarms in Android

The `AlarmManager` class lets you schedule app code to run at a future time, even if the app is not running:

- Used for reminders, scheduled tasks, periodic sync
- Types: `RTC` (Real-Time Clock), `RTC_WAKEUP`, `ELAPSED_REALTIME`, `ELAPSED_REALTIME_WAKEUP`
- `setExact()` - Fires at exact time (API 19+)
- `setRepeating()` - Fires repeatedly at intervals

```
AlarmManager alarmManager = (AlarmManager) getSystemService(Context.ALARM_SERVICE);
Intent intent = new Intent(this, AlarmReceiver.class);
PendingIntent pendingIntent = PendingIntent.getBroadcast(this, 0, intent, 0);

// Set alarm for 10 seconds from now
```

```
long triggerTime = SystemClock.elapsedRealtime() + 10 * 1000;  
alarmManager.setExact(AlarmManager.ELAPSED_REALTIME_WAKEUP, triggerTime,  
pendingIntent);
```

UNIT 6: Multithreading, AsyncTask & runOnUiThread

6.1 Threads in Android

Android apps run on the Main Thread (UI Thread) by default. Long operations on the main thread cause ANR (Application Not Responding) errors:

- Main Thread (UI Thread) - Handles all UI updates; never block this thread
- Worker Thread - Background thread for heavy tasks like network calls, database ops
- Only the Main Thread can update UI components

```
// Creating a background thread:
Thread thread = new Thread(new Runnable() {
    @Override
    public void run() {
        // Do heavy work here (network, database)
        // Cannot update UI here directly!
    }
});
thread.start();
```

6.2 AsyncTask Class

AsyncTask (deprecated in API 30, but still asked in exams) is a helper class to perform background operations and publish results on the UI thread:

Method	Thread	Purpose
onPreExecute()	UI Thread	Called before background task starts; show progress bar
doInBackground()	Background Thread	Perform the actual heavy task; call publishProgress()
onProgressUpdate()	UI Thread	Update UI with progress from doInBackground
onPostExecute()	UI Thread	Called after doInBackground; update UI with result

```
private class DownloadTask extends AsyncTask<String, Integer, String> {
    @Override
    protected void onPreExecute() {
        progressBar.setVisibility(View.VISIBLE);
    }

    @Override
    protected String doInBackground(String... urls) {
        // Simulate download
        for(int i=0; i<=100; i+=10) {
            publishProgress(i);
            Thread.sleep(500);
        }
        return "Download Complete";
    }

    @Override
    protected void onProgressUpdate(Integer... progress) {
        progressBar.setProgress(progress[0]);
    }

    @Override
```

```
        protected void onPostExecute(String result) {
            progressBar.setVisibility(View.GONE);
            Toast.makeText(MainActivity.this, result, Toast.LENGTH_SHORT).show();
        }
    }

    // Execute:
    new DownloadTask().execute("http://example.com/file");
```

6.3 runOnUiThread

`runOnUiThread()` is used to update UI from a background thread:

```
Thread thread = new Thread(() -> {
    // Background work...
    String result = fetchDataFromServer();

    // Update UI from background thread using runOnUiThread:
    runOnUiThread(() -> {
        textView.setText(result);
        progressBar.setVisibility(View.GONE);
    });
});
thread.start();
```

- **Alternative to AsyncTask**; simpler for quick UI updates from threads
 - **Handler and Looper** can also be used for thread-to-UI communication
-

UNIT 7: Data Storage in Android

7.1 Types of Storage in Android

Storage Type	Description	Best For
SharedPreferences	Key-value pairs stored as XML file; persistent across sessions	User settings, login state, small preferences
Internal Storage	Private file storage inside app's sandbox	Sensitive files; deleted when app uninstalls
External Storage	Public storage on SD card or shared external space	Large files, media, downloads that users can access
SQLite Database	Full relational database stored locally on device	Structured data with complex queries
Network/Cloud Storage	Data stored on remote server via REST API	Data shared across devices or users
Room Database	Abstraction layer over SQLite with annotations	Modern recommended approach for local DB

7.2 SQLite in Android

SQLite is a lightweight, embedded relational database used natively in Android:

SQLite Data Types:

- NULL - Null value
- INTEGER - Signed integer (1, 2, 3, 4, 6, or 8 bytes)
- REAL - Floating-point number (8-byte IEEE 754)
- TEXT - Text string encoded in UTF-8, UTF-16BE or UTF-16LE
- BLOB - Binary data stored as-is

Note: *SQLite uses dynamic typing (type affinity). Java int maps to INTEGER, String maps to TEXT, double maps to REAL.*

```
// Creating a database helper:
public class DBHelper extends SQLiteOpenHelper {
    private static final String DB_NAME = "MyDB";
    private static final int DB_VERSION = 1;

    public DBHelper(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE students (id INTEGER PRIMARY KEY, " +
            "name TEXT, marks REAL);");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldV, int newV) {
        db.execSQL("DROP TABLE IF EXISTS students");
        onCreate(db);
    }
}
```

```
}
```

7.3 JSON Parsing in Android

JSON (JavaScript Object Notation) is a lightweight data exchange format commonly used in REST APIs:

```
// Sample JSON:
// {"name":"Alice", "age":22, "subjects":["Math","Science"]}

try {
    JSONObject jsonObject = new JSONObject(jsonString);
    String name = jsonObject.getString("name");
    int age = jsonObject.getInt("age");
    JSONArray subjects = jsonObject.getJSONArray("subjects");
    for(int i=0; i<subjects.length(); i++) {
        Log.d("Subject", subjects.getString(i));
    }
} catch (JSONException e) {
    e.printStackTrace();
}

// Using Gson library (alternative):
Gson gson = new Gson();
MyClass obj = gson.fromJson(jsonString, MyClass.class);
```

What is Parsing? Parsing is the process of analyzing a string of text/data according to formal grammar rules and converting it into a structured format (like a Java object) that a program can use.

UNIT 8: Location Services, Maps & Sensors

8.1 Location Manager

The `LocationManager` class provides access to the device's location services:

- `getLastKnownLocation(provider)` - Returns last known location without requesting updates
- `requestLocationUpdates()` - Register for periodic location updates
- `removeUpdates()` - Stop receiving location updates
- `isProviderEnabled()` - Check if GPS or Network provider is enabled
- Providers: `GPS_PROVIDER` (accurate, uses GPS satellite), `NETWORK_PROVIDER` (uses WiFi/cell towers, less accurate but faster)

Note: Add permissions in Manifest: `ACCESS_FINE_LOCATION` (GPS) or `ACCESS_COARSE_LOCATION` (Network)

8.2 Google Maps in Android

- Use Google Maps Android API via Google Play Services
- Add API key in Manifest and Maps SDK dependency in `build.gradle`
- Use `SupportMapFragment` to embed map in layout
- `setZoomControlsEnabled(true)` - Displays zoom (+/-) controls on the map
- `addMarker()` - Add a marker/pin on the map
- `moveCamera()` / `animateCamera()` - Move map to a location
- `GoogleMap.MAP_TYPE_NORMAL`, `MAP_TYPE_SATELLITE`, `MAP_TYPE_TERRAIN`, `MAP_TYPE_HYBRID`

8.3 Accelerometer (Sensor)

The accelerometer measures acceleration forces applied to the device on three axes (X, Y, Z):

- Access via `SensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)`
- Implement `SensorEventListener` to receive sensor data
- Used for: shake detection, step counter, screen rotation, gaming
- `onSensorChanged(SensorEvent event)` - Called when new sensor data is available
- `event.values[0]` = X axis, `event.values[1]` = Y axis, `event.values[2]` = Z axis

8.4 PhoneGap Accelerometer

In PhoneGap/Cordova, the Accelerometer API is accessed through JavaScript:

```
// PhoneGap Accelerometer:
navigator.accelerometer.getCurrentAcceleration(
    function(acceleration) {
        alert('X: ' + acceleration.x + '\n' +
            'Y: ' + acceleration.y + '\n' +
            'Z: ' + acceleration.z);
    },
    function() { alert('Error getting acceleration'); }
);
```

8.5 Making Phone Calls in Android

```
// Permission in Manifest:
<uses-permission android:name="android.permission.CALL_PHONE"/>

// Making a call via implicit intent:
```

```
Intent callIntent = new Intent(Intent.ACTION_CALL);
callIntent.setData(Uri.parse("tel:9876543210"));
startActivity(callIntent);
```

```
// To just open dialer (no permission needed):
Intent dialIntent = new Intent(Intent.ACTION_DIAL);
dialIntent.setData(Uri.parse("tel:9876543210"));
startActivity(dialIntent);
```

UNIT 9: PhoneGap / Apache Cordova

9.1 What is PhoneGap?

PhoneGap (now Apache Cordova) is an open-source mobile development framework that allows building hybrid mobile apps using HTML, CSS, and JavaScript:

- Developed by Nitobi, acquired by Adobe, then donated to Apache as Cordova
- Write once, deploy everywhere - supports iOS, Android, Windows Phone
- Web app wrapped in a native WebView container
- Access native device features (Camera, GPS, Contacts, Accelerometer) via JavaScript plugins
- Not as fast as fully native apps

9.2 PhoneGap Plugins

PhoneGap plugins provide JavaScript interfaces to native device APIs:

- Camera Plugin - Access device camera to take photos or pick from gallery
- Geolocation Plugin - Access GPS coordinates
- Accelerometer Plugin - Access motion sensors
- Contacts Plugin - Read/write device contacts
- File Plugin - Access device file system
- InAppBrowser Plugin - Open URLs in an in-app browser
- Vibration Plugin - Control device vibration
- Network Information Plugin - Check network connectivity

Steps to Publish a Plugin to npm:

11. Create the plugin with proper plugin.xml, package.json, and source files
12. Ensure the plugin follows Cordova plugin structure and naming conventions
13. Test the plugin thoroughly on supported platforms
14. Create an account at npmjs.com if not already done
15. Run: npm login (enter username, password, email)
16. Run: npm publish in the plugin root directory
17. The plugin is now available for others to install via: cordova plugin add your-plugin-name

9.3 PhoneGap vs Native vs Web Applications

Feature	Native	Web App	Hybrid (PhoneGap)
Cross-platform	No	Yes (browser)	Yes (Write once, deploy anywhere)
Performance	Poorer performance compared to native	Depends on plugins for hardware access	UI does not always feel 100% native
Uses familiar web skills	HTML, CSS, JS	Depends on plugins for hardware access	Faster development time for simple apps
Cost-effective	Single codebase for multiple OS	Depends on plugins for hardware access	Large app size due to inclusion of native wrapper

9.4 How PhoneGap Applications Work

The developer creates an application using web technologies like HTML, CSS, and JavaScript. This web content is bundled into a native wrapper (WebView) provided by PhoneGap/Cordova. JavaScript plugins act as a bridge, allowing the web code to communicate with native device hardware APIs. The final app can then be installed on mobile devices just like a native application.

9.5 Structure of a PhoneGap Application

- **www/ folder:** Contains the web assets (index.html, styles.css, app.js).
- **config.xml:** Global configuration file for app name, icon, and permissions.
- **platforms/ folder:** Contains native project files generated for Android, iOS, etc.
- **plugins/ folder:** Stores installed plugins that provide access to native features.

9.6 Creating a PhoneGap Application - Steps

1. Install Node.js on the development machine.
2. Install Cordova CLI using npm: `npm install -g cordova`.
3. Create a new project: `cordova create hello com.example.hello HelloWorld`.
4. Navigate to the project folder: `cd hello`.
5. Add a target platform: `cordova platform add android`.
6. Add desired plugins: `cordova plugin add cordova-plugin-camera`.
7. Develop the UI/logic within the `www/` directory.
8. Build and run the app: `cordova run android`.

Note: The 'deviceready' event is crucial in PhoneGap — all device API access must happen after this event fires, confirming Cordova has fully loaded.

UNIT 10: Android Programming Examples

10.1 Registration Form with Validation

```
<!-- activity_register.xml -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText android:id="@+id/etName"
        android:hint="Full Name" android:inputType="textPersonName"
        android:layout_width="match_parent" android:layout_height="wrap_content"/>

    <EditText android:id="@+id/etEmail"
        android:hint="Email" android:inputType="textEmailAddress"
        android:layout_width="match_parent" android:layout_height="wrap_content"/>

    <EditText android:id="@+id/etPhone"
        android:hint="Phone" android:inputType="phone"
        android:layout_width="match_parent" android:layout_height="wrap_content"/>

    <EditText android:id="@+id/etPassword"
        android:hint="Password" android:inputType="textPassword"
        android:layout_width="match_parent" android:layout_height="wrap_content"/>

    <Button android:id="@+id/btnRegister"
        android:text="Register"
        android:layout_width="match_parent" android:layout_height="wrap_content"/>
</LinearLayout>

// RegisterActivity.java
public class RegisterActivity extends AppCompatActivity {
    EditText etName, etEmail, etPhone, etPassword;
    Button btnRegister;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_register);

        etName = findViewById(R.id.etName);
        etEmail = findViewById(R.id.etEmail);
        etPhone = findViewById(R.id.etPhone);
        etPassword = findViewById(R.id.etPassword);
        btnRegister = findViewById(R.id.btnRegister);

        btnRegister.setOnClickListener(v -> validateAndRegister());
    }

    private void validateAndRegister() {
        String name = etName.getText().toString().trim();
        String email = etEmail.getText().toString().trim();
        String phone = etPhone.getText().toString().trim();
        String password = etPassword.getText().toString().trim();

        if(name.isEmpty()) {
            etName.setError("Name is required"); return;
        }
    }
}
```

```

    }
    if(email.isEmpty() ||
!android.util.Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
        etEmail.setError("Valid email required"); return;
    }
    if(phone.length() != 10) {
        etPhone.setError("10-digit phone required"); return;
    }
    if(password.length() < 6) {
        etPassword.setError("Min 6 characters"); return;
    }
    Toast.makeText(this, "Registered Successfully!",
Toast.LENGTH_SHORT).show();
    }
}

```

10.2 Addition of Two Numbers

```

// MainActivity.java - Add two numbers
EditText et1 = findViewById(R.id.et1);
EditText et2 = findViewById(R.id.et2);
Button btnAdd = findViewById(R.id.btnAdd);
TextView tvResult = findViewById(R.id.tvResult);

btnAdd.setOnClickListener(v -> {
    int a = Integer.parseInt(et1.getText().toString());
    int b = Integer.parseInt(et2.getText().toString());
    tvResult.setText("Sum = " + (a + b));
});

```

10.3 Android Styles & Themes

Styles and Themes in Android allow defining consistent visual properties across the app:

- **Style** - A collection of attributes (color, font, size) applied to a single View
- **Theme** - A style applied to an entire Activity or Application

```

<!-- Define a style in res/values/styles.xml -->
<style name="MyButtonStyle" parent="Widget.AppCompat.Button">
    <item name="android:textColor">#FFFFFF</item>
    <item name="android:background">#2E75B6</item>
    <item name="android:padding">12dp</item>
    <item name="android:textSize">16sp</item>
</style>

<!-- Apply to a view -->
<Button style="@style/MyButtonStyle" ... />

<!-- Apply a theme to app in Manifest -->
<application android:theme="@style/Theme.MaterialComponents.Light.DarkActionBar">

<!-- Apply theme to specific Activity -->
<activity android:name=".MainActivity" android:theme="@style/MyTheme"/>

```

10.4 Alert Dialog

AlertDialog displays a dialog with a title, message, and action buttons:

```

AlertDialog.Builder builder = new AlertDialog.Builder(this);
builder.setTitle("Confirm");
builder.setMessage("Are you sure you want to delete?");
builder.setPositiveButton("Yes", (dialog, which) -> {
    Toast.makeText(this, "Deleted!", Toast.LENGTH_SHORT).show();
});
builder.setNegativeButton("No", (dialog, which) -> dialog.dismiss());

```

```
builder.setNeutralButton("Cancel", null);
AlertDialog dialog = builder.create();
dialog.show();
```

10.5 Rating Bar Control

RatingBar is a View that allows the user to rate something using star icons:

```
<!-- XML -->
<RatingBar
    android:id="@+id/ratingBar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:numStars="5"
    android:rating="3.5"
    android:stepSize="0.5"/>

// Java
RatingBar ratingBar = findViewById(R.id.ratingBar);
ratingBar.setOnRatingBarChangeListener((ratingBar1, rating, fromUser) -> {
    Toast.makeText(this, "Rating: " + rating, Toast.LENGTH_SHORT).show();
});
```

IMPORTANT QUESTIONS - EXAM PREPARATION LIST

Based on analysis of all 3 previous year question papers, these questions have high probability of appearing in your exam:

Q1 Type - Short Answers (2 marks each) - Most Repeated Topics

- List any six Android UI controls
- What is Dalvik Virtual Machine? Explain briefly
- What is Intent? List its types (Explicit and Implicit)
- What is a View in Android?
- What is an Android Fragment?
- List the types of ViewGroups / Layouts in Android
- What is Grid View?
- Name the types of menus in Android
- What is PhoneGap?
- Which class is used to create Notifications?
- List gesture methods used in touch events (onDown, onFling, onScroll, onLongPress, onDoubleTap, onSingleTapUp)
- List the data types in Android SQLite (NULL, INTEGER, REAL, TEXT, BLOB)
- What is the use of Alert Dialog in Android?
- What is the purpose of Android Alarms?
- List different mobile operating systems
- Write any two methods to access Location-based services

Q2 Type - Short Explanations (3 marks each) - High Probability

- Explain Toast with example (code)
- What is an Android Service? Explain types with examples
- What is mobile App? Explain types (Native, Web, Hybrid)
- What is Broadcast Receiver? Explain with example
- Explain Accelerometer in Android
- List five factors in developing Mobile Applications
- Explain LinearLayout with example
- What is parsing? Explain JSON parsing with code
- What is Android Virtual Device (AVD)?
- Write short note on Manifest File
- Write any two features of Android OS
- Describe PhoneGap Plugins concept
- Write properties of Spinner control

Q3 Type - Medium Answers (4 marks each) - Very Important

- Explain Android Activity Lifecycle with all callback methods
- Explain Android Architecture (all 5 layers) with diagram
- What are features of Android OS? List and explain
- Explain Content Providers with example and CRUD operations
- Write note on Android Gestures and touch events
- What is Thread? Explain use of AsyncTask class (all 4 methods)

- Explain various types of Mobile OS and differentiate them
- Describe multithreading in Android and how AsyncTask helps background processing
- Explain Services in Android with lifecycle diagram
- Explain Event Handling in Android (all methods)
- Explain PhoneGap plugins and steps to publish to npm
- Describe four factors in developing mobile apps

Q4 Type - Medium Answers (4 marks each) - Important

- Explain components of Android Architecture (Activity, Service, Broadcast Receiver, Content Provider)
- Describe Android Fragment lifecycle (all callback methods)
- What is Android Broadcast Receiver? Explain static and dynamic registration
- Explain Threads in Android (main thread, worker thread, runOnUiThread)
- Write Android program to print addition of two numbers
- Write short note on storage in Android (all types)
- Explain Activity Lifecycle in detail
- Explain ViewGroup class - explain any 4 layouts in detail
- Explain Event Handling in Android
- Explain Content Providers with how they are created and used
- Compare native, web, and hybrid applications

Q5 Type - Long Answers (5 marks each) - Must Prepare

- Create Android app to display subject list using Spinner and Buttons (XML + Java)
- Explain Android Notification with example (complete code)
- Write Java Android program for Registration Form with Validation (XML + Java)
- Explain Activity Lifecycle with diagram and example
- Discuss different types of Layouts in Android with examples
- Explain types of menus in Android and how to implement them using XML and Java

Q6 Type - Short Notes (5 marks each) - Important

- Manifest File - purpose, contents, structure with example
- PhoneGap Plugins - what they are, types, how to install and publish
- Types of Intent in Android - Explicit vs Implicit with code examples
- runOnUiThread - purpose, usage, code example
- Adapters in Android - types (ArrayAdapter, BaseAdapter, CursorAdapter, RecyclerView.Adapter)
- Event Handling in Android - listener types and implementation methods
- Android Architecture with diagram
- Android Styles and Themes
- Rating Bar Control - properties and implementation

Quick Revision: Key Terms to Remember

Term	One-line Definition
ART	Android Runtime - executes app code; replaced Dalvik in Android 5.0
DVM	Dalvik Virtual Machine - runs .dex files; register-based; one per app

Term	One-line Definition
APK	Android Package Kit - the app installation file format
Intent	Messaging object to start activities, services, or broadcast
Fragment	Reusable UI component embedded within an Activity
AsyncTask	Helper class for background tasks + UI updates (deprecated API 30)
ContentProvider	Shares structured data between apps using URI-based interface
Toast	Brief non-blocking popup message shown to user
Spinner	Drop-down widget for selecting one item from a list
Adapter	Bridge between data source and UI widget (ListView, Spinner)
ANR	Application Not Responding - occurs when main thread is blocked > 5 sec
AVD	Android Virtual Device - emulator for testing apps
PhoneGap	Framework to build hybrid apps using HTML/CSS/JS
Manifest	XML config file declaring all app components and permissions
Broadcast	System-wide message; Receiver listens and responds to broadcasts

All the Best for Your Exam!

Study this material thoroughly and you will score well in CS-552 MJ